

Técnicas de Segmentação de Imagens com Python: Um Estudo Comparativo entre Binarização de Otsu e Segmentação por Cor

Alcides Antonio Lorenski Neto¹

Murilo Morosini²

Resumo

A segmentação de imagens é uma etapa fundamental em sistemas de Visão Computacional, pois permite identificar e separar objetos de interesse do restante da imagem. Este trabalho apresenta um estudo teórico e prático sobre duas técnicas de segmentação amplamente utilizadas: a binarização de Otsu e a segmentação por cor no espaço HSV. O objetivo é analisar o funcionamento dessas técnicas, suas vantagens, limitações e aplicações em diferentes cenários. Para a implementação dos experimentos foi utilizada a linguagem Python em conjunto com a biblioteca OpenCV. Foram realizados testes com imagens digitais, comparando os resultados obtidos por cada método quanto à qualidade da segmentação, custo computacional e facilidade de implementação. Os resultados demonstram que a binarização de Otsu apresenta bom desempenho em imagens com contraste adequado, enquanto a segmentação por cor é mais eficiente quando os objetos possuem cores bem definidas e distintas do plano de fundo.

Palavras-chave: Visão Computacional. Segmentação de Imagens. Python. OpenCV. Binarização de Otsu. Segmentação por cor.

¹ Graduando do Curso de Ciência da Computação - Unoesc Campus São Miguel do Oeste. alcidesantonio08@hotmail.com

² Graduando do Curso de Ciência da Computação - Unoesc Campus São Miguel do Oeste. murilomorosini@gmail.com

1. INTRODUÇÃO

De acordo com Marengoni e Stringhini (2009, p. 126-127), a segmentação de imagens é uma das etapas mais importantes da Visão Computacional, sendo responsável por separar objetos de interesse do plano de fundo de uma imagem. Esse processo possibilita a separação de regiões de interesse na imagem, servindo como etapa fundamental para tarefas de reconhecimento de padrões e interpretação de imagens em sistemas de visão computacional.

Devido à grande quantidade de informações presentes em imagens digitais, técnicas de segmentação são amplamente utilizadas para reduzir a complexidade dos dados analisados. A segmentação consiste em dividir a imagem em regiões que representam estruturas ou objetos de interesse, separando-os do plano de fundo e facilitando etapas posteriores de processamento e análise (Rudek; Coelho; Junior, 2001 *apud* Russ, 1995; Liu, 2000).

A segmentação de imagens possui ampla aplicação em sistemas de visão computacional, sendo utilizada em atividades como controle de qualidade industrial, identificação e classificação de objetos, automação de processos produtivos, navegação de robôs e reconhecimento de padrões. Além disso, contribui para a redução do esforço computacional ao limitar a análise às regiões de interesse da imagem (Rudek; Coelho; Junior, 2001).

Neste trabalho são estudadas duas técnicas de segmentação de imagens amplamente utilizadas: a binarização de Otsu e a segmentação por cor no espaço HSV. Ambas permitem destacar objetos de interesse em uma imagem, porém utilizam abordagens distintas para realizar esse processo. A técnica de Otsu baseia-se na definição automática de um limiar de intensidade para separar objeto e fundo, enquanto a segmentação por cor utiliza intervalos de tonalidades para identificar regiões específicas da imagem.

Nos tópicos a seguir serão apresentados os fundamentos teóricos dessas técnicas, seus principais conceitos, vantagens e limitações, bem como exemplos práticos desenvolvidos na linguagem Python utilizando a biblioteca OpenCV. Ao final, será realizada uma análise comparativa dos métodos estudados, considerando aspectos como qualidade da segmentação, custo computacional, facilidade de implementação e adequação das aplicações.

2. DESENVOLVIMENTO

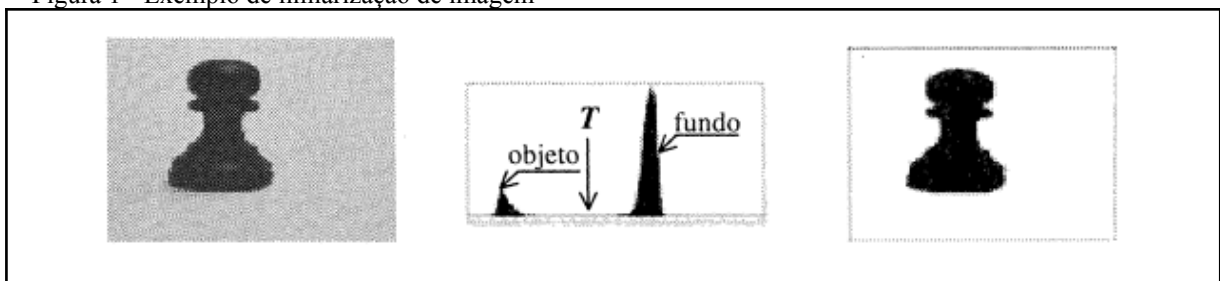
Segundo Artero e Tommaselli (2000, p. 39), dentre as técnicas de segmentação, a limiarização (*thresholding*) destaca-se pela sua simplicidade e eficiência em situações onde os

componentes da imagem apresentam níveis de brilho distintos. O princípio fundamental da operação consiste em definir um valor de corte, denominado limiar T , a partir do qual os pixels da imagem são classificados em duas classes: objeto e fundo.

A operação de limiarização transforma uma imagem em tons de cinza em uma imagem binária, atribuindo a cada pixel um valor conforme sua relação com o limiar estabelecido. Pixels cujos valores de brilho são inferiores a T recebem valor zero, enquanto aqueles com valores superiores são mantidos ou recebem valor máximo. Essa abordagem pressupõe que as classes presentes na imagem estão suficientemente separadas em termos de intensidade, de modo que um único valor de limiar seja capaz de distingui-las (Artero; Tommaselli, 2000, p. 39-40).

É possível analisar, na Figura 1, o valor de corte T e um processo de limiarização, no qual uma imagem original em tons de cinza é transformada em uma imagem binária. O limiar T é posicionado no vale entre esses dois picos, separando as duas classes. O resultado, apresentado à direita, demonstra a classificação dos pixels: aqueles com valor abaixo de T são convertidos para preto, preservando o objeto, enquanto os demais são convertidos para branco, eliminando o fundo.

Figura 1 - Exemplo de limiarização de imagem



Fonte: Artero e Tommaselli (2000, p. 40).

Os valores definidos para essa limiarização estão apresentados na Figura 2, na qual cada pixel de coordenadas (x, y) recebe valor 0 caso seu brilho seja inferior ao limiar T , ou valor 1 caso seja igual ou superior. Dessa forma, a imagem resultante é binária, ou seja, composta apenas por pixels pretos e brancos (Artero; Tommaselli, 2000, p. 41).

Figura 2 - Equação da limiarização

$$\text{Im}(x, y) = \begin{cases} 0 & \text{se } \text{Im}(x, y) < T \\ 1 & \text{se } \text{Im}(x, y) \geq T \end{cases}$$

Fonte: Artero e Tommaselli (2000, p. 41).

2.1 BINARIZAÇÃO DE OTSU

De acordo com Piyadasa (2022), a binarização de Otsu é uma técnica de limiarização global automática que converte uma imagem em tons de cinza em uma imagem binária (preto e branco). O método determina automaticamente um valor de limiar capaz de separar os pixels da imagem em duas classes distintas, correspondentes ao objeto e ao fundo, considerando que o histograma apresenta duas distribuições predominantes de níveis de intensidade.

Essa representação, exemplificada visualmente na Figura 1, resulta em uma imagem binária composta apenas pelos valores 0 e 255, que correspondem, respectivamente, aos níveis binários 0 e 1 (Piyadasa, 2022).

2.1.1 Funcionamento

O funcionamento do método de Otsu consiste em avaliar todos os possíveis valores de limiar presentes no histograma da imagem. Para cada valor analisado, os pixels são divididos em duas classes, sendo calculadas suas respectivas variâncias. Ao final, é selecionado o limiar que minimiza a variância intraclasse, proporcionando a melhor separação entre as classes da imagem (Piyadasa, 2022).

De acordo com Artero e Tommaselli (2000, p. 41) e Parker (2011, p. 142), inicialmente, são calculadas as probabilidades de ocorrência das duas classes, representadas por ω_0 e ω_1 , bem como suas respectivas médias dos níveis de cinza, μ_0 e μ_1 , conforme apresentado na Figura 3.

Figura 3 - Probabilidades e médias das classes

$$\omega_0 = \sum_{i=0}^L p_i \quad \omega_1 = 1 - \omega_0 \quad \mu_0 = \frac{\mu_t}{\omega_0} \quad \mu_t = \sum_{i=0}^L i \cdot p_i \quad p_i = \frac{n_i}{n}$$

Fonte: Parker (2011, p. 142).

Esses parâmetros são utilizados para determinar a variância entre as classes (σ^2_B), apresentada na Figura 4, que mede o grau de separação entre elas onde: ω_0 e ω_1 são as probabilidades das duas classes e μ_0 e μ_1 são as médias dos níveis de cinza de cada classe (Parker, 2011, p. 141).

Figura 4 - Variância entre as classes

$$\sigma_B^2 = \omega_0 \omega_1 (\mu_0 \mu_1)^2$$

Fonte: Parker (2011, p. 141).

Em seguida, nas Figuras 5 e 6 são empregadas para calcular a variância total da imagem (σ_T^2) e a média global dos níveis de cinza (μ_T), que representam a distribuição geral das intensidades presentes na imagem (Artero; Tommaselli, 2000, p. 41; Parker, 2011, p.141).

Figura 5 - Variância total da imagem

$$\sigma_T^2 = \sum_{i=0}^{l-1} (i - \mu_T)^2 \cdot p_i$$

Fonte: Artero e Tommaselli (2000, p. 41).

Figura 6 - Média global dos níveis de cinza

$$\mu_T = \sum_{i=0}^{l-1} i \cdot p_i$$

Fonte: Artero e Tommaselli (2000, p. 41).

Por fim, a Figura 7 estabelece o critério para a seleção do limiar ótimo por meio da razão entre a variância intraclasse e a variância total. Dessa forma, é escolhido o valor de limiar que maximiza essa relação, resultando na melhor discriminação entre as classes presentes no histograma (Artero; Tommaselli, 2000, p. 42).

Figura 7 - Critério de seleção do limiar

$$n = \frac{\sigma_B^2}{\sigma_T^2}$$

Fonte: Artero e Tommaselli (2000, p. 41).

Em termos simples, o método de Otsu analisa a distribuição dos níveis de cinza presentes no histograma da imagem e verifica diferentes valores de limiar. Para cada valor analisado, são calculadas as probabilidades e as médias das duas classes formadas, permitindo determinar o grau de separação entre elas. Em seguida, essa separação é comparada com a variabilidade total da imagem, sendo selecionado o limiar que proporciona a maior distinção entre as classes. Dessa forma, o método identifica automaticamente o valor mais adequado para realizar a binarização da imagem.

2.1.2 Vantagens e Desvantagens

Segundo Piyadasa (2022), a binarização de Otsu apresenta como principal vantagem a determinação automática do valor de limiar, eliminando a necessidade de ajustes manuais e tornando o processo mais simples e objetivo. Além disso, o método busca maximizar a separação entre o objeto de interesse e o fundo da imagem por meio da análise do histograma, produzindo resultados satisfatórios quando há boa distinção entre as classes. Em imagens com iluminação uniforme, a técnica é capaz de gerar imagens binárias nítidas, preservando adequadamente os detalhes relevantes.

Como desvantagem, tal modelo utiliza um único limiar para toda a imagem, caracterizando-se como uma técnica de limiarização global. Dessa forma, seu desempenho pode ser comprometido em imagens que apresentam variações de iluminação, sombras ou reflexos, uma vez que diferentes regiões podem exigir limiares distintos para uma segmentação adequada. Nesses casos, o método pode produzir resultados menos precisos, ocasionando perda de informações ou classificação incorreta de determinadas regiões da imagem (Piyadasa, 2022).

2.1.3 Código-Fonte

O Código 1 apresenta a implementação da binarização de Otsu utilizando a linguagem Python e a biblioteca OpenCV. Inicialmente, a imagem é carregada e convertida para tons de cinza, etapa necessária para a aplicação do método. Em seguida, a função *threshold()* é utilizada com a flag *THRESH_OTSU*, permitindo que o valor de limiar seja calculado automaticamente a partir do histograma da imagem. Por fim, a imagem resultante é exibida utilizando a biblioteca *Matplotlib* para análise visual dos resultados obtidos.

Codigo 1 - Código-fonte de Otsu

```
1 import cv2
2 from matplotlib import pyplot as plt
3
4 img = cv2.imread('img.jpg')
5 img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
6
7 _, otsu_thresholding = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY+cv2.THRESH_OTSU)
8
9 plt.figure(figsize = (10,10))
10 plt.imshow(otsu_thresholding, 'gray')
11 plt.show()
```

Fonte: Piyadasa (2022).

2.1.4 Resultados da Segmentação por Método Otsu

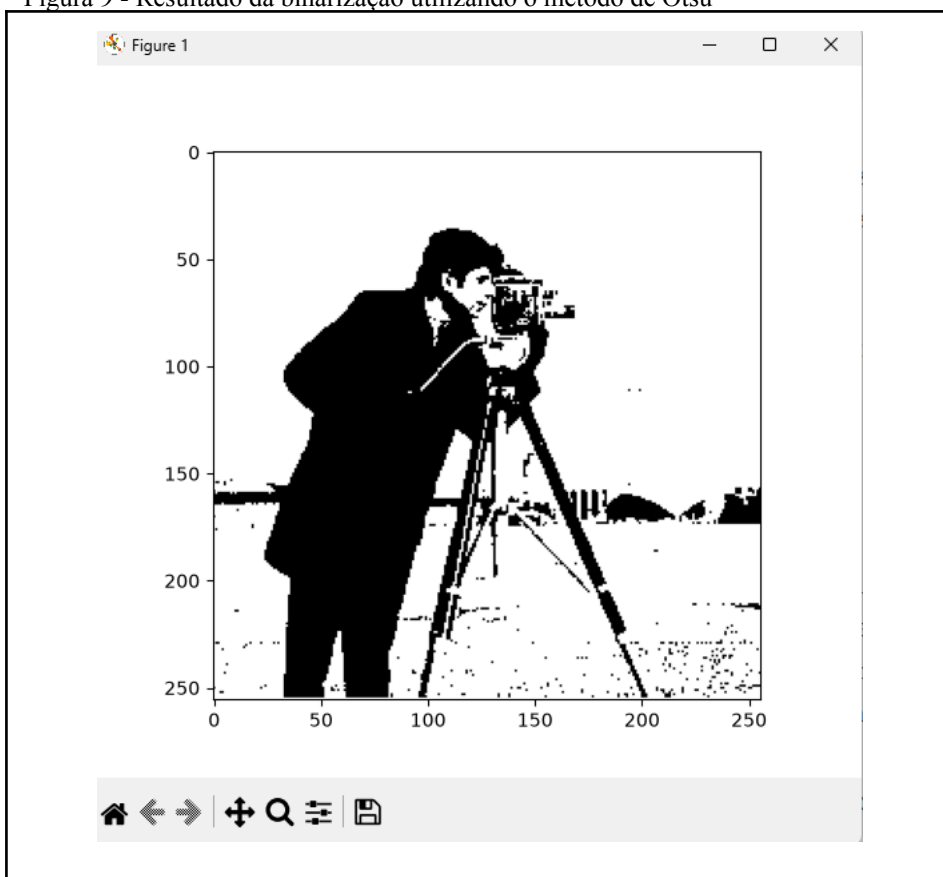
Para avaliar o desempenho da binarização de Otsu, foi utilizada uma imagem contendo informações textuais sobre um fundo de intensidade distinta. A Figura 8 apresenta a imagem original utilizada no experimento, enquanto a Figura 9 mostra o resultado obtido após a aplicação da técnica de segmentação.

Figura 8 - Imagem original utilizada no experimento



Fonte: Piyadasa (2022).

Figura 9 - Resultado da binarização utilizando o método de Otsu



Fonte: Os autores (2026).

Ao analisar os resultados, observa-se que o método foi capaz de determinar automaticamente um valor de limiar adequado para separar o texto do plano de fundo. Como consequência, os pixels pertencentes ao objeto de interesse foram destacados, gerando uma imagem binária composta apenas por pixels pretos e brancos. O resultado obtido está de acordo com o funcionamento descrito anteriormente, demonstrando a capacidade do método de maximizar a separação entre as classes presentes na imagem. O método foi capaz de destacar adequadamente a silhueta do fotógrafo e do tripé, preservando as principais características da cena.

Porém, a qualidade da segmentação depende das características da imagem analisada, podendo ser afetada por variações de luz, sombras ou baixo contraste entre objeto e fundo.

2.2 SEGMENTAÇÃO POR COR

A segmentação por cor é uma técnica que consiste em identificar e isolar regiões de uma imagem com base nas características cromáticas dos pixels, classificando-os de acordo com intervalos de tonalidade previamente definidos. Diferente da binarização de Otsu, que

age sobre a intensidade luminosa de imagens em tons de cinza, a segmentação por cor explora a informação cromática da imagem para separar objetos de interesse do plano de fundo.

2.2.1 Funcionamento

Para compreender o funcionamento da segmentação por cor, é necessário primeiramente entender o espaço de representação cromática utilizado. Em visão computacional, as cores são representadas de acordo com o espaço de cor em que a imagem se apresenta, sendo o RGB e o HSV os espaços de cor mais comuns, em ambos, para representar uma cor específica, é necessária a combinação de três componentes.

Em sistemas de visão computacional e processamento de imagens, o espaço HSV é um dos principais modelos utilizados para a representação de cor. A representação de uma imagem colorida no espaço HSV permite um grau de separação entre as cores e a iluminação. Diferentemente do espaço RGB, o espaço HSV não utiliza canais para representação de cores primárias, mas sim para representar a matiz (*hue*), a saturação (*saturation*) e o valor (*value*). (Benevides, 2022).

Os três canais que compõem o espaço HSV possuem significados distintos e complementares. De acordo com Lima (2020), o canal H (*hue*) consiste na cor propriamente dita; o canal S (*saturation*) corresponde à saturação de cor, muito relacionada à intensidade; e o canal V (*value*) representa o brilho da cor. Com as variações dessas características, chegam-se a diferentes níveis de cores, com intensidades e brilhos distintos.

Souza (2020) explica que ao trabalhar com segmentação, o espaço HSV se torna mais adequado do que o RGB, pois neste último, para modificar o brilho de uma cor por exemplo, praticamente todos os três canais precisam ser alterados. Já no HSV, a cor ou tonalidade é definida pelo canal H, sendo possível aumentar ou diminuir o brilho de uma cor somente alterando o canal V (*value*).

Corroborando com essa perspectiva, o espaço de cores HSV é mais adequado para a detecção de cores em imagens, pois separa a informação de cor (H) da informação de luminosidade (V) e de saturação (S), o que permite que a detecção de cores seja mais robusta a variações de iluminação. (Inspier, 2024).

No OpenCV, os valores dos canais do espaço HSV são mapeados de forma específica para adequação à representação em 8 bits. Conforme Inspier (2024), o canal *Hue* (H), que no espaço HSV padrão varia de 0 a 360 graus, é mapeado no OpenCV para o intervalo de 0 a 180, de modo que para converter um valor de matiz do espaço padrão para o utilizado pela

biblioteca, é necessário dividir o valor por 2 ($H/2$). Os canais *Saturation* (S) e *Value* (V), que no espaço padrão variam de 0% a 100%, são mapeados para o intervalo de 0 a 255

Quanto aos componentes cromáticos, Monolito Nimbus (2025) apresenta a distribuição das cores no canal de matiz: o Matiz (H) varia de 0° (vermelho) a 60° (amarelo), 120° (verde), 180° (ciano), 240° (azul) e 300° (magenta); a Saturação (S) vai de 0% (sem cor) a 100% (com cor); e o Valor (V) de 0% (preto) a 100% (branco)

O processo de segmentação por cor no espaço HSV compreende três etapas principais. A primeira consiste na conversão da imagem do espaço BGR para o espaço HSV por meio da função *cv2.cvtColor()*. A segunda consiste na definição dos intervalos de cor desejados e na criação de uma máscara binária com a função *cv2.inRange()*. Segundo Rosebrock (2014), essa função retorna uma máscara binária, em que os valores iguais a 1 (branco) indicam os pixels cujos valores estão dentro do intervalo definido, e os valores iguais a 0 (preto) indicam pixels fora desse intervalo. A terceira etapa consiste na aplicação da máscara sobre a imagem original por meio da operação *cv2.bitwise_and()*, que, ainda segundo Rosebrock (2014), mantém cada pixel da imagem original caso o valor correspondente na máscara seja igual a 1.

2.2.2 Vantagens e Desvantagens

Conforme Zeepalm (2023), a principal vantagem do espaço de cores HSV é que ele separa os detalhes de cor e brilho, o que permite que um objeto mantenha a mesma aparência cromática independentemente das variações de iluminação ou da quantidade de luz refletida por ele, o que é particularmente útil em aplicações do mundo real, onde as condições de iluminação podem variar drasticamente.

Além disso, a segmentação por cor apresenta baixo custo computacional, uma vez que a criação e a aplicação da máscara envolvem operações simples e diretas sobre os canais da imagem. Montagner (2017) ressalta que a segmentação de cores no espaço HSV pode ser realizada com um simples intervalo no canal H, e, para evitar a detecção de variantes muito escuras ou claras, podem ser adotados valores mínimos para os canais de saturação e valor.

Como desvantagem, a definição dos intervalos de cor exige ajuste manual e conhecimento prévio das características cromáticas do objeto de interesse. Esses valores variam conforme a imagem analisada, tornando o processo dependente de experimentação e calibração. GeeksforGeeks (2024) observa que testar os limites HSV em diferentes condições de iluminação e utilizar barras de controle em tempo real pode ajudar a refinar a abordagem de detecção de cores.

Outra limitação relevante diz respeito à segmentação de cores localizadas nas extremidades do círculo cromático. Conforme Insper (2024), para cores como o vermelho, que ocupa as bordas do espaço de cores HSV, uma abordagem viável consiste em realizar duas seleções de intervalos separadas para cada faixa, aplicar a segmentação em ambas e, posteriormente, unir as duas máscaras resultantes por meio do operador lógico *OR* do OpenCV, que retorna branco se pelo menos um dos pixels for branco.

Por fim, a técnica pode apresentar falhas quando o objeto possui cores semelhantes às do plano de fundo ou de outros elementos da imagem, dificultando sua correta segmentação.

2.2.3 Código-Fonte

O Código 2 apresenta a implementação da segmentação por cor utilizando Python e a biblioteca OpenCV. Inicialmente, a imagem é carregada e convertida do espaço de cores BGR para o HSV. Em seguida, são definidos os limites inferior e superior da faixa de cor desejada, e a função `cv2.inRange()` é utilizada para criar a máscara binária. Por fim, a máscara é aplicada sobre a imagem original com a função `cv2.bitwise_and()`, e os resultados são exibidos com a biblioteca Matplotlib.

Código 2 - Código-fonte da segmentação por cor no espaço HSV.

```

1  import cv2
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # Carrega a imagem
6  image = cv2.imread('imagem.jpg')
7
8  # Converte de BGR para HSV
9  hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
10
11 # Define os limites inferior e superior da cor desejada (ex: azul)
12 lower = np.array([100, 50, 50])
13 upper = np.array([130, 255, 255])
14
15 # Cria a máscara binária
16 mask = cv2.inRange(hsv, lower, upper)
17
18 # Aplica a máscara sobre a imagem original
19 result = cv2.bitwise_and(image, image, mask=mask)
20
21 # Exibe os resultados
22 plt.subplot(1, 3, 1)
23 plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
24 plt.title('Original')
25 plt.subplot(1, 3, 2)
26 plt.imshow(mask, cmap='gray')
27 plt.title('Máscara HSV')
28 plt.subplot(1, 3, 3)
29 plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
30 plt.title('Segmentada')
31 plt.show()

```

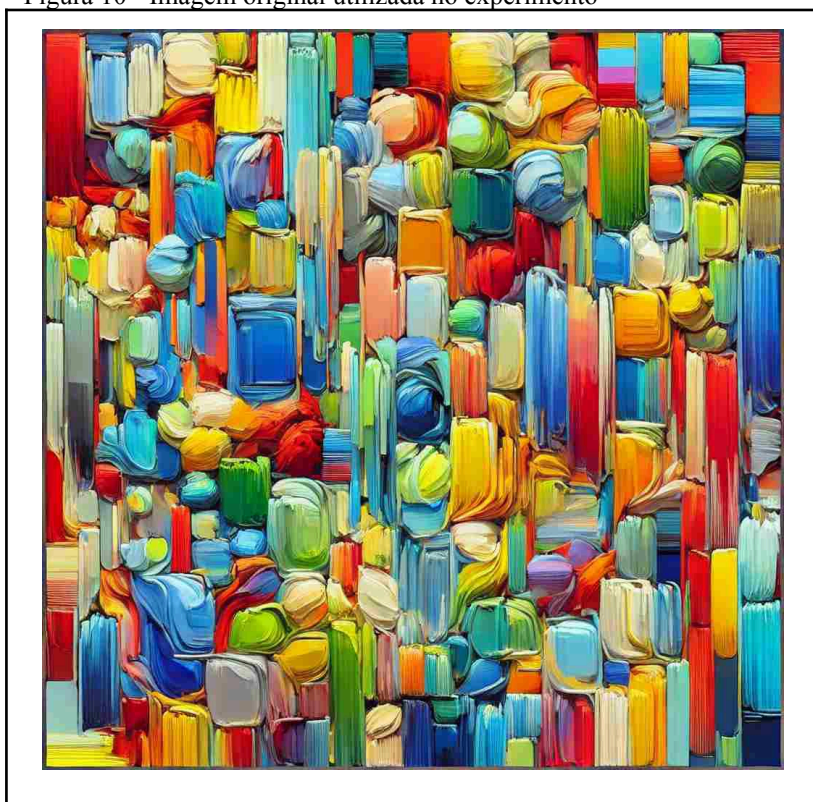
Fonte: Os autores (2026), adaptado de OpenCV Documentation (2024) e Souza (2020).

2.2.4 Resultados da Segmentação por Segmentação por Cor

Para avaliar o desempenho da segmentação por cor no espaço HSV, foi utilizada uma imagem de composição pictórica abstrata contendo uma ampla variedade de cores, com presença simultânea de tons de vermelho, amarelo, verde, azul e laranja, distribuídos de forma heterogênea por toda a extensão da cena. Essa característica torna a imagem especialmente adequada para demonstrar a capacidade da técnica de isolar uma cor específica mesmo em cenários de alta complexidade cromática.

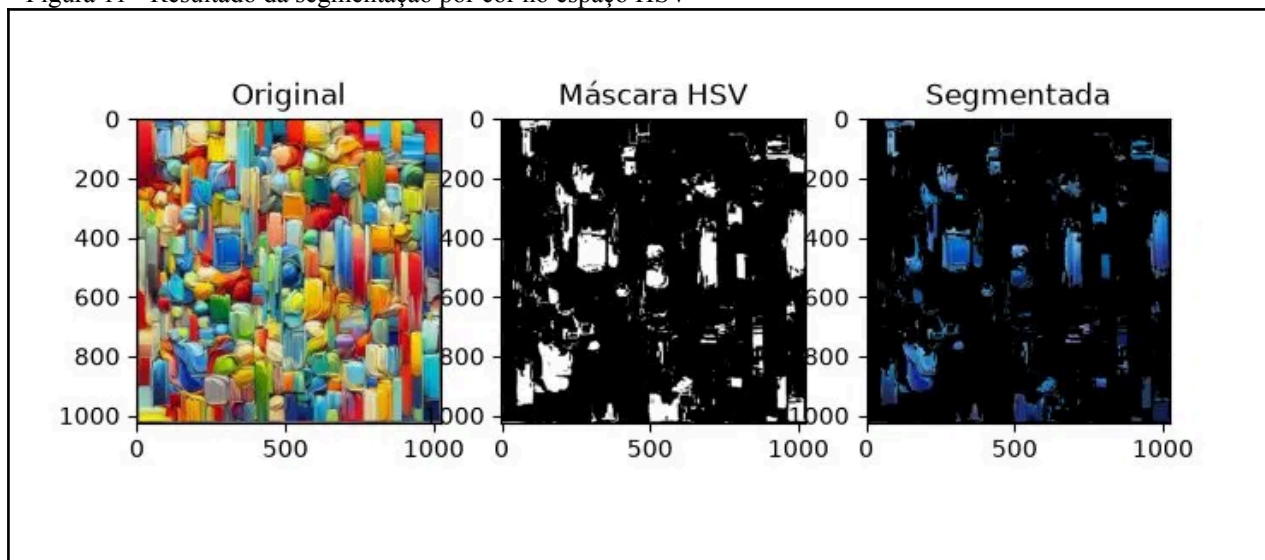
Para o experimento, foram definidos os limites inferior $[100, 50, 50]$ e superior $[130, 255, 255]$ no espaço HSV, correspondentes à faixa de tonalidade azul. A Figura 10 apresenta a imagem original utilizada no experimento, enquanto a Figura 11 exibe, lado a lado, a imagem original, a máscara binária gerada pela função *cv2.inRange()* e o resultado final da segmentação obtido com a função *cv2.bitwise_and()*.

Figura 10 - Imagem original utilizada no experimento



Fonte: GeeksforGeeks (2024).

Figura 11 - Resultado da segmentação por cor no espaço HSV



Fonte: Os autores (2026).

Ao analisar os resultados, observa-se que o método foi capaz de isolar com eficácia as regiões de tonalidade azul presentes na imagem, descartando as demais cores. A máscara binária gerada evidencia os pixels pertencentes à faixa de cor definida, representando-os em branco e suprimindo o restante em preto. O resultado final preserva apenas as regiões azuis sobre fundo preto, demonstrando a precisão do método na identificação de cores específicas. A qualidade da segmentação está diretamente condicionada à precisão dos limites HSV estabelecidos e às condições de iluminação da cena, podendo ser necessário ajuste dos parâmetros quando a imagem apresentar variações de brilho ou sombras.

2.3 ANÁLISE COMPARATIVA DAS TÉCNICAS

A binarização de Otsu e a segmentação por cor no espaço HSV são técnicas amplamente utilizadas na segmentação de imagens, porém apresentam características distintas que influenciam diretamente sua aplicação. A escolha entre uma ou outra depende principalmente da qualidade da imagem, das informações disponíveis e do objetivo da segmentação.

Em relação à qualidade da segmentação, a binarização de Otsu apresenta melhores resultados quando existe um contraste bem definido entre o objeto e o plano de fundo. Nesses casos, o método consegue separar adequadamente as duas regiões por meio de um limiar calculado automaticamente. Entretanto, quando a imagem possui sombras, reflexos ou variações de iluminação, a qualidade da segmentação pode ser reduzida. Já a segmentação por

cor no espaço HSV tende a produzir resultados mais precisos em imagens coloridas, principalmente quando o objeto possui uma tonalidade característica diferente das demais regiões da cena. Dessa forma, consegue isolar com eficiência elementos específicos mesmo em imagens visualmente complexas.

Quanto ao custo computacional, ambas as técnicas apresentam baixo consumo de recursos e podem ser utilizadas em aplicações que exigem processamento rápido. A binarização de Otsu necessita da análise do histograma da imagem para determinar o limiar ideal, enquanto a segmentação por cor realiza operações simples de conversão para o espaço HSV e aplicação de máscaras. Assim, os dois métodos possuem desempenho computacional satisfatório, sendo adequados para aplicações em tempo real.

No aspecto da facilidade de implementação, a binarização de Otsu apresenta vantagem por exigir poucas etapas de processamento. Basta converter a imagem para tons de cinza e aplicar o algoritmo para que o limiar seja calculado automaticamente. Por esse motivo, sua implementação é considerada simples e rápida. Em contrapartida, a segmentação por cor requer a definição manual dos intervalos HSV correspondentes à cor desejada. Esse processo geralmente demanda testes e ajustes para cada imagem ou ambiente analisado, tornando sua implementação um pouco mais trabalhosa.

Em relação aos cenários mais adequados para cada técnica, a binarização de Otsu é indicada para aplicações em que a informação relevante está associada à intensidade dos pixels, como digitalização de documentos, reconhecimento de textos, leitura de códigos e segmentação de objetos em imagens com alto contraste. Já a segmentação por cor é mais apropriada para situações em que a cor é a principal característica do objeto de interesse, como rastreamento de objetos coloridos, sistemas de inspeção visual, identificação de peças em linhas de produção e aplicações de robótica e visão computacional.

Portanto, observa-se que a binarização de Otsu se destaca pela simplicidade e pela definição automática do limiar, sendo mais adequada para imagens em tons de cinza com contraste bem definido. Por outro lado, a segmentação por cor no espaço HSV oferece maior capacidade de discriminação em imagens coloridas, desde que seja possível definir adequadamente os intervalos de cor desejados.

3. CONCLUSÃO

Neste trabalho foram estudadas duas técnicas de segmentação de imagens: a binarização de Otsu e a segmentação por cor no espaço HSV. Ambas demonstraram ser

eficientes para destacar objetos de interesse em imagens, cada uma apresentando características adequadas para diferentes cenários de aplicação. Além da pesquisa teórica, a implementação prática em Python utilizando a biblioteca OpenCV permitiu compreender o funcionamento dos métodos e comparar seus resultados. Dessa forma, o trabalho contribuiu para ampliar o conhecimento sobre Visão Computacional e a importância da segmentação de imagens em sistemas computacionais.

Entre os principais aprendizados obtidos, destacam-se a compreensão do processo de segmentação de imagens, a utilização da biblioteca OpenCV para processamento digital de imagens e a análise das diferenças entre técnicas baseadas em limiarização e segmentação por cores. Como dificuldades encontradas, destacam-se a escolha adequada dos parâmetros para a segmentação por cor no espaço HSV, a necessidade de realizar ajustes para diferentes imagens e a interpretação dos resultados gerados pelos algoritmos.

Esses desafios contribuíram para um melhor entendimento das limitações e aplicações práticas das técnicas estudadas.

REFERÊNCIAS

ARTERO, Almir Olivette; TOMMASELLI, Antonio Maria Garcia. Limiarização automática de imagens digitais. **Boletim de Ciências Geodésicas**, Curitiba, v. 6, n. 1, p. 38-48, 2000. Disponível em: <https://revistas.ufpr.br/bcg/article/view/63040/36904>. Acesso em: 04 jun. 2026.

MARENGONI, Maurício; STRINGHINI, Denise. Tutorial: Introdução à Visão Computacional usando OpenCV. **Revista de Informática Teórica e Aplicada**, v. 16, n. 1, p. 125–160. Disponível em: https://seer.ufrgs.br/index.php/rita/article/view/rita_v16_n1_p125/7289. Acesso em: 04 jun. 2026.

PARKER, J. R. **Algorithms for Image Processing and Computer Vision**. 2. ed. Indianapolis: Wiley Publishing, Inc., 2011. Disponível em: http://kiwi.bridgeport.edu/cpeg585/Algorithms_for_Image_Processing_and_Computer_Vision.pdf. Acesso em: 11 jun. 2026.

PIYADASA, Tharinda. Image Binarization in a Nutshell. **Medium**. 05 jun. 2022. Disponível em: <https://medium.com/@tharindad7/image-binarization-in-a-nutshell-b40b63c0228e>. Acesso em: 11 jun. 2026.

RUDEK, Marcelo; COELHO, Leandro dos Santos; JUNIOR, Osiris Canciglier. **VISÃO COMPUTACIONAL APLICADA A SISTEMAS PRODUTIVOS: FUNDAMENTOS E ESTUDO DE CASO**. 2001. Disponível em: https://www.researchgate.net/profile/Osiris-Canciglieri-Junior/publication/228549623_Visao_Computacional_Aplicada_a_Sistemas_Produtivos_Fundamentos_e_Estudo_de_Caso/links/00b49537cba93253ff000000/Visao-Computacional-Aplicada-a-Sistemas-Produtivos-Fundamentos-e-Estudo-de-Caso.pdf. Acesso em: 04 jun. 2026.

BENEVIDES, Thiago. Começando com OpenCV e Python. **LinkedIn Pulse**, 02 mai. 2022. Disponível em: <https://pt.linkedin.com/pulse/come%C3%A7ando-com-opencv-e-python-thiago-benevides>. Acesso em: 16 jun. 2026.

GEEKSFORGEEEKS. Choosing the correct upper and lower HSV boundaries for color detection with cv::inRange (OpenCV). **GeeksforGeeks**, 26 set. 2024. Disponível em: <https://www.geeksforgeeks.org/computer-vision/choosing-the-correct-upper-and-lower-hsv-boundaries-for-color-detection-with-cv-inrange-opencv/>. Acesso em: 16 jun. 2026.

INSPER. Segmentação de imagens — Robótica Computacional. **Insper**, 2024. Disponível em: https://insper.github.io/robotica-computacional/modulos/04-visao-p1/atividades/5-segmentacao_2/. Acesso em: 11 jun. 2026.

LIMA, Suzana S. V. M. Visão computacional: detecção de cores em tempo real, utilizando Python e OpenCV. **Medium**, 22 mai. 2020. Disponível em: <https://suzana-svm.medium.com/vis%C3%A3o-computacional-detec%C3%A7%C3%A3o-de-cores-em-tempo-real-utilizando-python-e-opencv-a466444d40e>. Acesso em: 11 jun. 2026.

MONOLITO NIMBUS. Modelos de cores HSV e LAB. **Monolito Nimbus**, 03 jan. 2025. Disponível em: <https://www.monolitonimbus.com.br/modelos-de-cores-hsv-e-lab/>. Acesso em: 16 jun. 2026.

MONTAGNER, Igor dos Santos. Espaços de cores. **Blog Igor Montagner**, 19 mai. 2017. Disponível em: <http://igormontagner.blogspot.com/2017/05/espacos-de-cores.html>. Acesso em: 16 jun. 2026.

OPENCV DOCUMENTATION. Changing colorspace. OpenCV, 2024. Disponível em: https://docs.opencv.org/4.13.0/df/d9d/tutorial_py_colorspaces.html. Acesso em: 11 jun. 2026.

ROSEBROCK, Adrian. OpenCV and Python Color Detection. **PyImageSearch**, 04 ago. 2014. Disponível em: <https://pyimagesearch.com/2014/08/04/opencv-python-color-detection/>. Acesso em: 16 jun. 2026.

SOUZA, Otávio. Segmentando objetos pela cor | OpenCV | Python. **Medium**, 10 jun. 2020. Disponível em: <https://offsouza.medium.com/segmentando-objetos-pela-cor-opencv-487d5181b473>. Acesso em: 11 jun. 2026.

ZEEPALM. Object detection from image and video using HSV color space. **ZeePalm**, 2023. Disponível em: <https://www.zeepalm.com/blog/object-detection-from-image-and-video-using-hsv-color-space>. Acesso em: 16 jun. 2026.